A simulated approach to experimental analysis of tradeoffs between hard and soft cohesion caching strategies

Simon Gilbu
Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
simon.gilbu@uit.no

Thomas Tekie
Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
thomas.tekie@uit.no

I. Introduction

In this paper, we take a simulated experimental approach to comparing the cost and tradeoffs in a `GET/SET` system. The aim is to assess the tradeoffs between hard and soft cohesion caching in a reproducible manner that is independent of underlying systems.

We evaluate two policies for caching, where caching is used as service acceleration [1]. These are push-based eviction and TTL, and build our assumptions around small write-heavy services in web applications. Our underlying assumption throughout this work is that certain systems might be more interested in "fresh" data [2], that is, data for which a cache returns the most recently written value, and we try to highlight some of the potential costs in terms of cache-hit ratios and a metric of "usefulness" that we argue carries significance for such systems.

II. Related work

We built on our previous work, which is available in the GitHub repository along with the source files for our experiments and simulations. Further, several articles exist, for instance, surveys, case studies, and theoretical foundations for evaluating cache behavior in different systems [1], [3], [4]. We complement existing research in this field by providing both an overview of the specific question relating to concrete "costs" of hard cohesion, evaluating a metric considering both overall cache hit ratios and the "correctness" of retrieved data, and by providing reproducible results through simulation.

III. Terms and Definitions

A `GET/SET` system is some system in which we can arbitrarily store data using a `GET` or a `SET` operation. Examples include SQL databases, NoSQL databases, maps, dictionaries, off-the-shelf caching solutions like Memcached, and storage APIs.

With soft cohesion we refer to systems that are eventually consistent with the underlying storage, but that need not be consistent at any given time. this means that our soft cohesion system actively works towards becoming consistent with some "source of truth" but provides no guarantees. A system with hard cohesion is one in which active work is performed to ensure that we are always consistent with the source of truth. that is hard cohesion gives us a promise about "correctness" with respect to the underlying data it is representing.

In our simulation there are two types of storage: the cache and the underlying storage. Our underlying storage is some arbitrary `GET/SET` system that will always be correct. When a `SET` operation is performed, we perform that operation directly on our storage.

When a `GET` operation is performed, this is performed first towards our cache. The cache then returns whatever entry it holds, or defers to storage if no valid entry is found. This deferring to storage is a **cache miss**; the inverse is a **cache hit**. Our cache-hit ratio is defined as the number of cache hits over all `GET` operations.

Each item is stored alongside some key, and can be retrieved using that same key. We consider the "correctness" of our data to be a binary question: if a `GET` operation on some key provides the most recent data that was supplied in a `SET` operation, our data is correct. A cache hit that returns correct data is called a fresh hit. The ratio of fresh hits over all cache hits is considered the **freshness** of a system.

The **usefulness** of our system is the share of all `GET` operations that are answered correctly from the cache, that is, the number of fresh hits over the number of `GET` operations. It is calculated as FRESHNESS * HIT-RATIO.

We mention both the terms 'GET/SET' and use the actionable verbs of "read" and "write". It is important to note that while these are in practice identical, we use 'GET' to refer to

any arbitrary extraction of state in an abstract system, and the term ”read” to specifically talk about retrieving concrete data from some storage mechanism in a given system. The same logic applies to the terms ‘SET‘ and ”write”.

A write-heavy service is assumed to be a service in which at least 30% of the `GET` and `SET` operations are `SET` operations [3].

A. Cold and warm cache

A cache is *cold* when it is empty, so the first read of every key is a miss that has to go to storage. A cache is *warm* when it already holds entries from earlier requests, so reads can be answered from the cache from the start. A long-running system is mostly warm, because the cold start happens only once. Every run in our experiments starts with a cold cache, and all reads are counted, so the hit ratios include the cold start.

IV. Experimental Setup

Our simulation has a core, which is a `GET/SET` API with a cache in front of it. The experiment works in three steps: we generate a trace of requests in advance, feed it to the core, and log the result of each request as it happens.

The entire experiment is a collection of generating traces, benchmarking each trace and generating the relevant reporting, before putting together our results, calculating our relevant metrics and presenting them in a final result table. Shown in figure1.

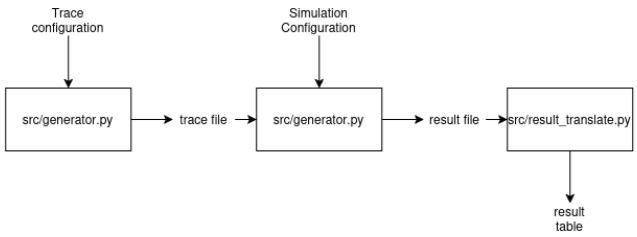


Fig. 1. Experimental procedure

A. Simulation Core

Our simulation core represents an API receiving requests to either **read** or **write** (`GET` or `SET`) some data to a given key. We assume that the core has an arbitrary keyspace in the form of a given integer numerical value. That is, any whole integer functions as a valid key, and the core simulation does not impose any restrictions on what number is or isn't a valid key, nor do we impose that there is a limited number of keys permissible.

We provide the core with a **throughput rate**, which is the number of requests it is assumed to simulate per second. Every time our simulation processes a request, it keeps track of the simulated time by incrementing a “timer”, which is a floating-point number representing the number of seconds passed. This allows us to simulate any given rate of throughput without regard to the real runtime of our simulation.

The core reads an input file line by line, in the format `OPERATION KEY VALUE`, and performs the given operation. On a `SET` or write operation, it will inform the cache of eviction for the given key (eviction can be ignored, as is the case for TTL) before it writes the key to the underlying storage. On a `GET` or read operation, the API first asks the cache for the given key. If the cache returns `None`, indicating that the key was not found, the storage is asked to retrieve the key. When a key is retrieved from storage it is also written into the cache, so that the cache can hold the relevant data next time the cache is asked for it.

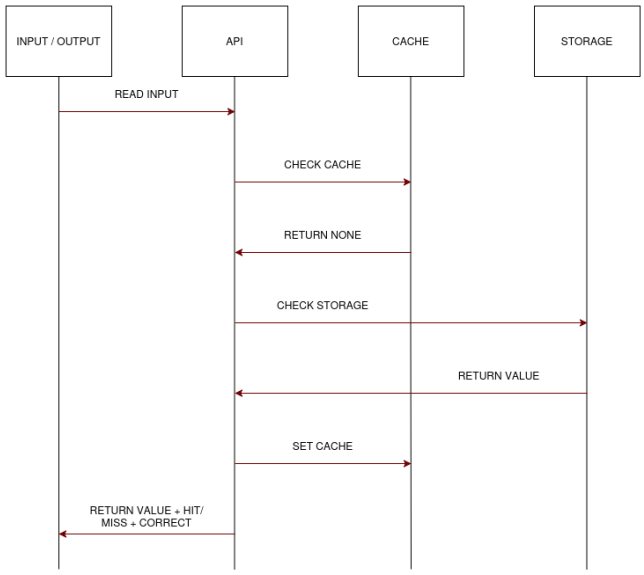


Fig. 2. Illustration of a cache miss

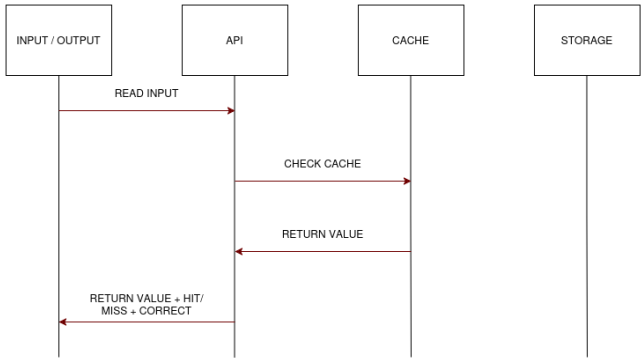


Fig. 3. Illustration of a cache hit

As shown in Figure 2 and Figure 3, a cache hit fully

avoids touching the underlying storage. Our simulation aims to "behave as if" this was the case; however, for the purposes of reporting correctness it is important to note that we in reality compare the value in our cache with that of the underlying storage. This means that the behavior we aim to represent through our simulation is not accurate to the behavior our simulation is performing, but the overall data flow does in the general case show the equivalent properties with regard to what data is in the cache.

TTL Caching: Our simulation is run using two caching strategies, one of which is the familiar TTL cache [1], [5]. In TTL caching a cache entry is valid for a set duration and has some temporal state. When we execute a `GET` operation towards a TTL cache we also provide a simulated time. If a key is found, we compare the current time according to our simulation with the expiration of the cache entry. If the cache entry has an expiration less than the simulated time, we discard the entry and return a `None` value, indicating that the cache no longer has any data relevant to our key.

In a `SET` operation our TTL cache enters data into the cache, alongside a timestamp for when this cache entry expires, which is defined to be the current simulation time plus some lifetime indicating how long the cache entry is valid after entry. We previously mentioned that our API sends an `EVICT` request on write. The TTL cache implements this method for polymorphism, such that the API can be cache agnostic, but does not provide any functionality within the method. Calling `EVICT` for some key on our TTL cache does not modify the state of the cache in any way.

PushEvict Caching: This is our hard cohesion caching strategy. Keys do not expire, and any passed variable relating to time is considered irrelevant for this strategy. Instead, it keeps all keys in the cache until they are manually evicted by the API, which happens when the API executes a `SET` or write against the underlying storage which triggers the *EVICT*.

B. Trace Generation

In order to generate reproducible traces, we precompute these from fixed seeds. A generator can be found in `src/generator.py` in the GitHub repository associated with our work. The generator accepts two primary access patterns, a write ratio given as a floating-point number in the range [0..1], a maximum integer for valid keys, a specified number of accesses or requests to generate, and a random seed to use for generation.

Our two access patterns, or distributions, relate to the distribution of keys and can be either `RANDOM` or `ZIPF`. Random selects keys at random in the range [0..KEYMAX], where each key has a `1/KEYMAX` chance of being selected. Zipf follows a power-law distribution in which the chance for any key to be selected is given by `w / tw`, where w is the weight of a single key and tw is the total weight of all keys. Further, w is given as `w(n) = 1/(n+1)` for each `n` in [0..KEYMAX], and `w(0) = 1`.

This can be observed by figures: 5 and 4.

C. Warmup

We do not use a separate warmup period. Every run starts with an empty cache, so the cold start is part of what we measure and all reads are counted. A cold start causes at most one extra miss per key, the first time the key is read. A trace has at most N such misses. A trace has 1 000 000 accesses, of which a share $1 - w$ are reads, where w is the write ratio. The share of reads that can be cold misses is at most $N/((1 - w) \cdot 1\,000\,000)$. For $N = 10\,000$ this is 1.4% at $w = 0.3$ and 3.3% at $w = 0.7$. For $N = 1\,000$ it is 0.14% and 0.33%, and for $N = 100$ it is 0.014% and 0.033%.

D. Experimental protocol

We compare TTL and PushEvict on the same traces, and here every trace is generated from these values.

TABLE I
EXPERIMENT CONFIGURATION PARAMETERS

Parameter	Value
Write ratio	0.3 and 0.7
Keymax values	100, 1 000, 10 000
Accesses per trace	1 000 000
Throughput	50 000, 100 000, 200 000
Distribution	Random and Zipf
Seed	1
TTL (simulated seconds)	5

Runs

- Each trace is run with two strategies, TTL and PushEvict, at three throughput values, which gives $3 \cdot 2 = 6$ runs per trace. With 12 traces (2 distributions, 2 write ratios and 3 keyspaces), this makes 72 runs in total.

Procedure

For every trace, a Bash script (`tracer.sh`) does the following:

1) `generator.py` writes the trace for the chosen number of keys, distribution, write ratio and seed.
2) `benchmark.py` replays the trace for TTL and PushEvict at each throughput and prints one log line per request.
3) `counter.py` reads the log and computes the hit ratio as the reads with `CACHE_HIT=True` divided by all reads, and the freshness as the hits with `CACHE_VALID=True` divided by all hits.
4) The script appends one line per run to `results.txt` and deletes the trace.

`results_translate.py` then groups the lines by strategy and prints a result table, as shown in the appendix. PushEvict does not depend on time, so its three runs per trace must give identical results, which they do. This is also a useful check that the simulation is deterministic.

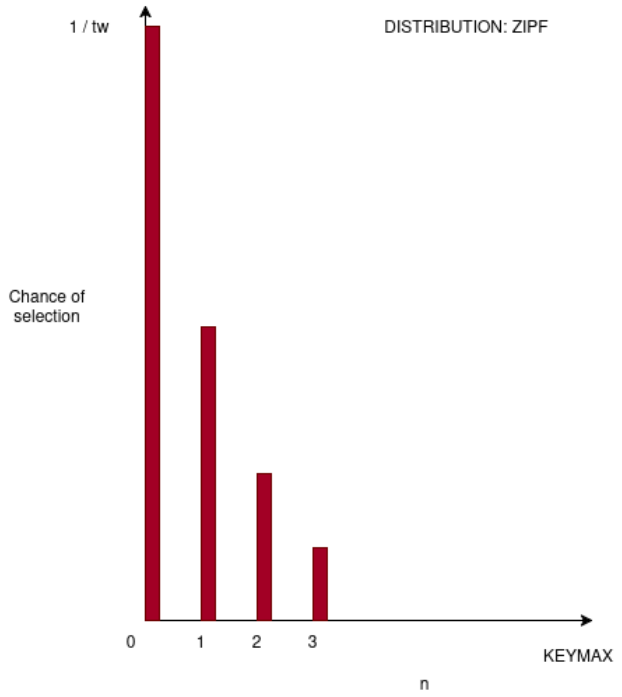


Fig. 4. Relative weight of keys following the Zipf distribution pattern

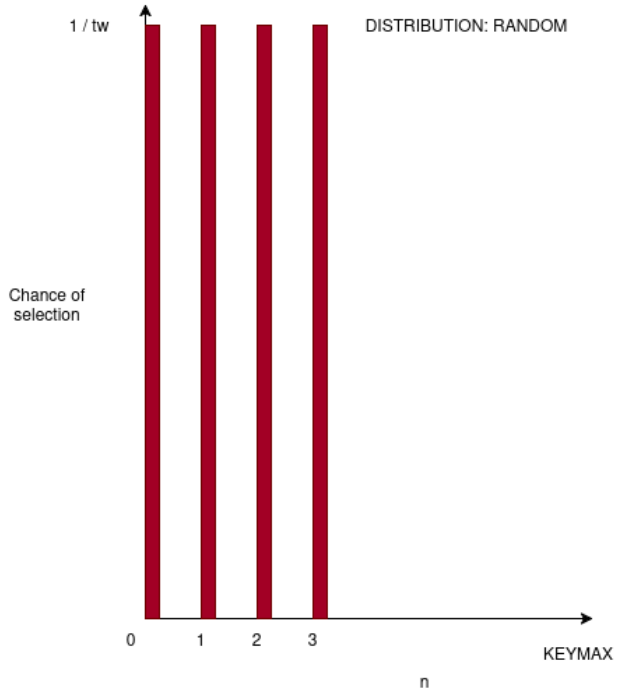


Fig. 5. Relative weight of keys following the random distribution pattern

V. Results

Table II in the appendix shows the hit ratio, the freshness and the usefulness for both strategies and both key distributions, at write ratios 0.3 and 0.7. For TTL there is one row per window: 250 000, 500 000 and 1 000 000 requests and the TTL window is the throughput times the TTL, which gives 250 000, 500 000 and 1 000 000 requests. The usefulness is the hit ratio times the freshness.

A. TTL lifetimes and keyspace

For limited keyspaces, where `keymax = 100`, we see that our TTL cache has very similar performance when measured in percentage points of "usefulness". For distributions favoring a smaller subset of keys, following our power-law distribution, we notice a similar behavior with Zipf as in the point above.

Looking at keyspaces of 10 000 keys, we see that as our TTL duration increases, the freshness of a key decreases.

Comparing keyspaces of 10 000 keys for TTL caches between the *random* and *zipf* distributions, we notice that our summary shows a higher usefulness in the *random* case.

We assume that, given the goal of optimizing for usefulness in a TTL caching system with a high ($> 30\%$) write ratio, it would be optimal to prioritize short lifetimes over a large subset of keys to reduce the number of `READ`s of any given key, and then enable some mechanism that prevents any one key or subset of keys from being the concentrated target of `READ` operations.

Or in other terms: a rapidly changing small set of data will be poorly represented by a system utilizing time-based invalidation for cohesion if we care about retrieving the most recent data.

B. TTL as a means of reducing access delays

- The worst hit ratio for a TTL cache seems to occur given a `random` distribution with a 70% write ratio over a space of 10 000 keys, and sits at $\sim$85.5%.
- The best hit ratio for a PushEvict cache seems to occur given a `random` or `zipf` distribution with a 30% write ratio over 100 keys, and sits at $\sim$70%.
- Hit ratios for TTL caches increase when the keyspace is dense or the accessed keys are clustered by a power-law distribution (`zipf`).
- Hit ratios for TTL caches seem to increase in relation to the lifetime of the TTL cache entry.

Given the above points, we make the assumption that TTL caching is favorable if we care about reducing the overall delay to a service, provided that access delays can be expressed as

$$(\text{hit ratio} \cdot \text{cache delay}) + ((1 - \text{hit ratio}) \cdot \text{storage delay}).$$

That is, if the cache delay provides a significant benefit over accessing the underlying storage and we only care about our delay, a long-lived TTL entry is assumed to be optimal over the PushEvict strategy.

C. PushEvict and usefulness

We note that usefulness for a PushEvict strategy will, given the limitations of our simulation, be equivalent to its hit ratio.

- The hit ratio and usefulness of a PushEvict strategy do not show signs of significant change as the keyspace changes.
- The hit ratio and usefulness of a PushEvict strategy do not show signs of significant change as the pattern of access (`random` or `zipf`) changes.
- The hit ratio and usefulness of a PushEvict strategy seem to be correlated with the write ratio of the underlying system.
- The usefulness of a PushEvict system is in the worst case just below 30%. This is the case for a write ratio of 0.7.
- The best-case usefulness for a TTL strategy is $\sim$13.2% given a keyspace of 10 000, a write ratio of 0.3 and a `random` access pattern with a window of 250 000.

This indicates to us that while the PushEvict strategy is highly dependent on the write ratio of a system, it will provide a higher degree of usefulness than a TTL caching strategy would, assuming we care about ensuring that our data aligns with the values present in the underlying storage. Further, a 30% hit ratio could provide some level of improvement in terms of access times, given the previously stated formula, if we assume that the cache delay is less than the storage delay.

VI. Discussion

There are a handful of limiting factors that reduce the applicable scope of our work. While these factors don't necessarily decrease the merit of the overarching patterns we see given the context of our simulation, they are definitely factors worth considering both for future work and when attempting to translate the results into decisions regarding real world systems.

A. Real system impacts

A real world implementation of server side in-memory caching as a service-accelerator [1] could presumably run for periods of times in the order of months [3], with varying loads of traffic during this period. if that is the case our TTL example is no longer representative, our simulation assumes a continuous stream of requests in which the processing time is equal for every request regardless of a cache hit or a cache miss. further we have defined our configuration values for TTL lifetimes in regards to the total duration of our simulation. setting them at 25%, 50% and 100% respectively. These values help indicate a pattern of decreasing freshness as the TTL increases, but does not shed light on what happens with relatively short lifetimes in services. if for instance a cache entry was valid for 3 seconds in a cache system that ran for several months with "high traffic".

B. Simulated time

Simulated time does not clearly and accurately represent real time, that is to say when evaluating the feasibility of a caching scheme such as PushEvict against TTL our measurements alone are not sufficient to state that one provides a clear benefit in terms of latency compared to the other. Access-delay for underlying storage, and the time to read from and populate the cache, may differ across several services [6] and these times must be accounted for in each individual case in order to gain a clear picture of what the presented data means for any given system.

C. Keyspaces and "keymax"

We use the term keymax to determine the largest allowed key, and state that all keys exist in the range *[0..KEYMAX]*. Real world systems might provide other mechanisms such as hashing to allow arbitrary keys, and could be bound by either a configuration-provided maximum capacity or utilize an entire systems available storage to provide lookups for given keys [7]. That is to say the size of a systems storage and its underlying mechanism for determining how many entries it permits might differ drastically from our assumptions.

D. PushEviction and the promise of freshness

We assume that requests are sequential and ordered, in particular this might be challenging our notion of freshness for the PushEvict strategy. In particular that is the case for any system processing requests with some degree of concurrency or parallelism. For example; if a service is available over a network the ordering of network requests might not be arriving in the order they were sent [8]. In this situation we could imagine a scenario where:

- A sends a WRITE
- B sends a READ
- B gets a HIT
- A's write is committed and the cache is updated

In a sequence such as that above we can no longer claim that B receives a cache-value that reflects the latest issued write. From A's perspective, B received a hit with stale data. This could skew the meaning of "usefulness" in certain environments.

VII. Limitation

- Synthetic workload: The traces follow simple random and zipf distributions with a fixed write ratio.
- Single seed: We have not repeated the runs with other seeds, so we cannot say how much the numbers vary.
- No warmup: Every run starts with an empty cache and all reads are counted, so our hit ratios include the cold start as supposed to describing a warm cache.

VIII. Further work

It is assumed that we could increase the usefulness of a hard cohesion caching strategy further if instead of PushEviction we performed a PushWrite, in which we not evict the cache-data but instead overwrite it with the updated values to ensure that our cache is synchronized against our storage. This would be dependent on timing-factors that might be specific to a given system or subsets of systems.

IX. Conclusion

In this paper, we discuss the implementation, experiments and results from simulating and comparing two different caching strategies in order to better outline the tradeoffs between hard and soft cohesion. We explain our metric of "usefulness" and showcase how it correlates not only with the cache-hit ratio but also with the data stored in cache and how it relates to the underlying storage.

We discuss our results and evaluate systems in which either cohesion strategy is useful, and provide our source-code, including the simulator and experiments in a public GitHub repository.

References

[1] J. Wang, "A survey of web caching schemes for the Internet," *ACM SIGCOMM Computer Communication Review*, vol. 29, no. 5, pp. 36–46, Oct. 1999. DOI: 10.1145/505696.505701

[2] S. An and Y. Cao, "Making cache monotonic and consistent," *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 891–904, 2022. DOI: 10.14778/3574245.3574271 [Online]. Available: https://www.vldb.org/pvldb/vol16/p891-cao.pdf

[3] J. Yang, Y. Yue, and K. V. Rashmi, "A large-scale analysis of hundreds of in-memory key-value cache clusters at Twitter," *ACM Transactions on Storage*, vol. 17, no. 3, pp. 1–35, Aug. 2021. DOI: 10.1145/3468521

[4] M. Umar and B. Tripathi, *Experimental analysis of server-side caching for web performance*, arXiv preprint arXiv:2602.06074, Feb. 2026. [Online]. Available: https://arxiv.org/abs/2602.06074

[5] J. Gwertzman and M. Seltzer, "World-wide web cache consistency," in *Proceedings of the USENIX 1996 Annual Technical Conference*, San Diego, CA, Jan. 1996. [Online]. Available: https://www.cs.fsu.edu/~awang/courses/cop5611_s2022/www_consistency.pdf

[6] J. Dean, *Numbers everyone should know*, From the talk "Designs, Lessons and Advice from Building Large Distributed Systems", LADIS 2009; transcribed by Brendan O'Connor, Accessed: November 2026, 2009. [Online]. Available: https://brenocon.com/dean_perf.html

[7] Memcached, *Memcached user guide: FAQ*, Official documentation, Accessed: November 2026. [Online]. Available: https://docs.memcached.org/userguide/faq/

[8] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978. DOI: 10.1145/359545.359563 [Online]. Available: https://lamport.azurewebsites.net/pubs/time-clocks.pdf

Appendix

TABLE II
COMPLETE RESULTS. HIT, FRESH AND USEFUL ARE IN PERCENT. WINDOW IS THE NUMBER OF REQUESTS BEFORE A TTL ENTRY EXPIRES.

Dist.	Writes	Keys	Strategy	Window	Hit	Fresh	Useful
random	0.30	100	PushEvict	–	70.0	100.0	70.0
			TTL	250 000	99.9	1.2	1.2
			TTL	500 000	99.9	1.1	1.1
			TTL	1 000 000	100.0	1.1	1.1
random	0.30	1 000	PushEvict	–	69.7	100.0	69.7
			TTL	250 000	99.1	1.4	1.4
			TTL	500 000	99.4	0.8	0.8
			TTL	1 000 000	99.5	0.4	0.4
random	0.30	10 000	PushEvict	–	67.0	100.0	67.0
			TTL	250 000	91.0	14.6	13.2
			TTL	500 000	93.8	7.1	6.7
			TTL	1 000 000	95.2	3.5	3.3
random	0.70	100	PushEvict	–	29.9	100.0	29.9
			TTL	250 000	99.9	1.1	1.1
			TTL	500 000	99.9	1.1	1.0
			TTL	1 000 000	100.0	1.0	1.0
random	0.70	1 000	PushEvict	–	30.0	100.0	30.0
			TTL	250 000	98.5	0.7	0.7
			TTL	500 000	99.2	0.4	0.4
			TTL	1 000 000	99.5	0.2	0.2
random	0.70	10 000	PushEvict	–	29.2	100.0	29.2
			TTL	250 000	85.5	6.5	5.5
			TTL	500 000	91.9	3.1	2.8
			TTL	1 000 000	95.3	1.5	1.4
zipf	0.30	100	PushEvict	–	70.0	100.0	70.0
			TTL	250 000	99.9	1.1	1.1
			TTL	500 000	99.9	1.1	1.1
			TTL	1 000 000	100.0	1.1	1.0
zipf	0.30	1 000	PushEvict	–	69.7	100.0	69.7
			TTL	250 000	99.1	1.4	1.4
			TTL	500 000	99.4	0.8	0.8
			TTL	1 000 000	99.5	0.4	0.4
zipf	0.30	10 000	PushEvict	–	67.1	100.0	67.1
			TTL	250 000	92.5	8.2	7.6
			TTL	500 000	94.2	5.4	5.1
			TTL	1 000 000	95.3	3.3	3.1
zipf	0.70	100	PushEvict	–	30.0	100.0	30.0
			TTL	250 000	99.9	1.1	1.1
			TTL	500 000	99.9	1.0	1.0
			TTL	1 000 000	100.0	1.0	1.0
zipf	0.70	1 000	PushEvict	–	29.8	100.0	29.8
			TTL	250 000	98.5	0.7	0.7
			TTL	500 000	99.2	0.4	0.4
			TTL	1 000 000	99.5	0.2	0.2
zipf	0.70	10 000	PushEvict	–	29.1	100.0	29.1
			TTL	250 000	89.8	3.8	3.4
			TTL	500 000	92.9	2.5	2.3
			TTL	1 000 000	95.2	1.5	1.4